

COURSE DESIGN • 前期工作补充

机票抓取与自动更新系统

以当前代码实现为基准，整理产品流程图、低保真产品原型图和可行性分析，用于课程报告正文和答辩 PPT 截图。

前端

Vue 3 / Vite /
Element Plus /
ECharts

后端

Spring Boot
3.3.5 / JDBC /
REST API

爬虫

Scrapy /
PyMySQL /
Docker
Compose 调度

数据库

MySQL: 航班、采集任务、
收藏、搜索历史

一、产品流程图

业务闭环

围绕普通用户、管理员和爬虫三条主线组织流程，形成“采集数据、查询展示、用户操作、任务追踪”的完整闭环。

用户端查询与收藏

User

1 登录/注册

- 1 保存 token 与用户信息，进入受保护页面。

2 筛选航班

- 2 选择出发地、到达地、日期、数据源、航空公司等条件。

3 查看结果

- 3 后端查询 flight 表，返回航班列表并记录搜索历史。

4 详情与价格

- 4 查看航班详情和 flight_price_snapshot 价格历史。

5 收藏管理

- 5 写入或删除 favorite 表，收藏页展示关注航班。

管理端采集任务

Admin

1 查看数据源状态

- 1 确认 API Key、爬虫命令和数据源配置是否可用。

2 提交采集参数

- 2 填写数据源、出发地、到达地、日期、乘客数和结果数。

3 后端调度爬虫

- 3 Spring Boot 通过 CrawlService 调用 Docker Compose。

4 爬虫采集入库

- 4 Scrapy 获取数据，pipeline 清洗、校验、写入 MySQL。

5 任务状态回显

- 5 更新 crawl_job 状态、成功条数、失败条数和完成时间。



二、产品原型图

原型图采用低保真线框表达页面结构和功能入口，页面范围与当前 Vue 路由和组件实现保持一致。

登录 / 注册页

航班查询工作台

收藏与搜索历史

采集任务管理

数据源状态页

航班详情与价格历史

三、可行性分析

围绕当前已实现系统，从技术、经济、操作、数据、时间和风险六个角度判断项目是否适合作为课程设计交付。

技

技术可行

Vue、Spring Boot、Scrapy、MySQL 均为成熟开源技术，当前代码已经形成前端、后端、爬虫、数据库闭环。

经

经济可行

课程开发主要依赖本地环境和开源组件，除外部数据 API Key 外无明显额外成本。

操

操作可行

普通用户和管理员页面职责清晰，适合按登录、查询、同步、收藏、管理任务顺序演示。

数

数据可行

flight、crawl_job、price_snapshot、favorite、search_history 等表可以支撑当前主线功能。

时

时间可行

核心功能已经落地，后续重点是演示数据、报告整理、截图和答辩材料制作。

险

风险可控

外部接口、环境启动和数据量不足是主要风险，可通过样例数据和演示前验证降低影响。

主要风险	可能影响	应对方式
外部 API 未配置	采集任务无法返回真实数据	提前准备样例数据，并通过数据源状态页说明配置情况
本地依赖较多	演示启动链路较长	按 MySQL、后端、前端、爬虫顺序提前验证
数据量不足	列表和图表展示效果弱	演示前手动触发同步或导入样例航班数据

说明：AI/chat 模块在当前代码中保留，但本图册以当前主线功能为准，仅将其作为后续扩展方向，不纳入核心流程。